

Assignment-1

Q1. Explain the difference between frontend, backend, and full-stack development with suitable real-world examples.

Ans. Frontend, backend, and full-stack development are three major areas of web development, each handling different parts of how a website or web application works.

Frontend development focuses on the part of a website that users can see and interact with — the visual design, layout, buttons, menus, animations, and overall user experience. It is also called *client-side* development because it runs on the user's browser. Frontend developers mainly use technologies like **HTML**, **CSS**, and **JavaScript**, along with frameworks such as **React**, **Angular**, or **Bootstrap**. For example, on a website like **YouTube**, everything visible to the user — such as the search bar, video thumbnails, and like buttons — is part of the frontend.

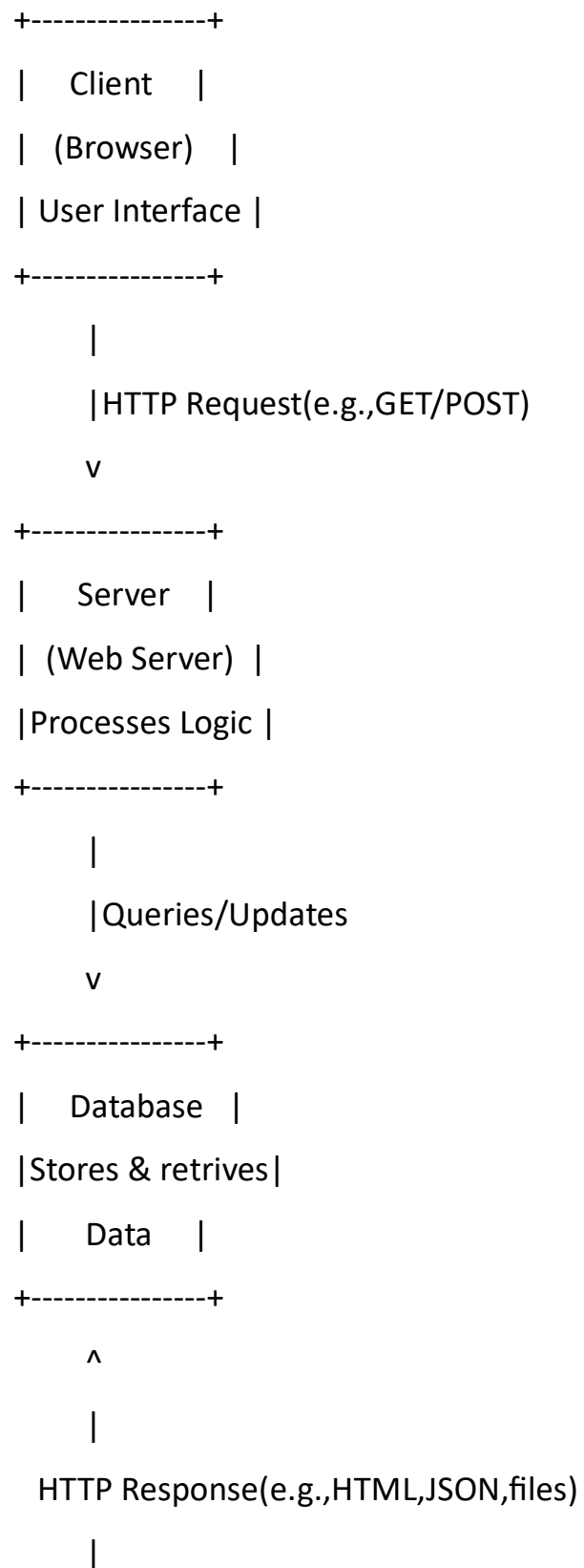
Backend development, on the other hand, is the *server-side* part of a website. It deals with how the website works behind the scenes — managing data, user authentication, and communication between the server and the database. Backend developers use languages such as **Python**, **Java**, **Node.js**, or **PHP**, and work with databases like **MySQL** or **MongoDB**. For example, when you log in to YouTube, the backend verifies your username and password, stores your preferences, and fetches your watch history from the database.

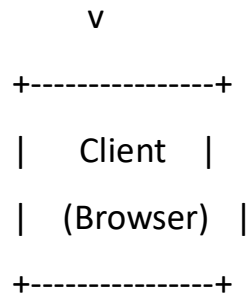
A **full-stack developer** is someone who can work on both the frontend and backend parts of a website. They build complete web applications from start to finish, handling both what users see and how the system works internally. A common example of a full-stack setup is the **MERN stack** (MongoDB, Express, React, Node.js). For instance, a full-stack developer could build an entire platform like **Netflix** — designing the interface users interact with (frontend) while also creating the server, database, and API connections that store and deliver content (backend).

In simple terms, frontend is like the **front of a restaurant** where customers eat, backend is the **kitchen** where food is prepared, and full-stack developers manage **both the dining area and the kitchen**, ensuring everything works smoothly together.

2. Create a simple diagram showing how the client-server model works in web architecture.

Ans.





Q3. Describe how a browser requests and displays a web page from a web server.

Ans. When you type a website URL (like `www.example.com`) in your browser and press Enter, the browser starts a **series of steps** to fetch and display the web page:

1. **DNS Lookup:**

The browser first translates the human-readable URL into an **IP address** using the **Domain Name System (DNS)**. This tells the browser which server on the internet it needs to contact.

2. **Sending an HTTP Request:**

Once the IP address is known, the browser opens a **TCP connection** to the web server and sends an **HTTP request** (or HTTPS for secure websites). This request usually asks for a specific resource, like `index.html`.

3. **Server Processing:**

The web server receives the request. If the page is static, the server directly sends the HTML file. If it is dynamic, the server may run backend code (like PHP, Node.js, or Python scripts) and query a database to generate the HTML.

4. **Sending an HTTP Response:**

The server sends the **HTTP response** back to the browser. This response contains:

- **HTML:** the structure of the page
- **CSS:** styling information
- **JavaScript:** scripts for interactivity
- Other resources like images, fonts, or videos

5. **Rendering the Page:**

The browser receives the response and starts rendering the page:

- Parses the HTML and builds the **DOM (Document Object Model)**
- Applies CSS to style the elements
- Executes JavaScript to make the page interactive

- Loads external resources like images, videos, and fonts

6. **Displaying to the User:**

After parsing and executing all content, the browser displays the **fully rendered web page** for the user to see and interact with.

Q4. Identify and list the tools required to set up a web development environment. Explain the purpose of each.

Ans. **Table**

Tool	Purpose
Text Editor / IDE	Write, edit, and debug code efficiently
Web Browser	Test and view web pages
Version Control System (Git)	Track code changes and collaborate
Web Server / Local Server	Run web applications locally
Package Manager (npm, yarn)	Install and manage dependencies
Frameworks / Libraries	Speed up development with reusable components
Database (MySQL, MongoDB)	Store and manage application data
Terminal / Command Line	Run commands, manage files, and control servers
Browser Developer Tools	Inspect, debug, and optimize web pages
Hosting / Collaboration Platforms	Store code online and enable team collaboration

Q5. Explain what a web server is and give examples of commonly used servers.

Ans. A **web server** is a computer system or software that stores, processes, and delivers web pages to users over the internet. When a user types a URL in a browser, the browser sends an **HTTP request** to the web server, which responds with the requested resources such as HTML files, CSS, JavaScript, images, or dynamic content generated by backend scripts. Web servers host websites, process client requests, serve static and dynamic content, and manage security and access control. Commonly used web servers include **Apache HTTP Server** (open-source and widely used), **Nginx** (lightweight and high-performance), **Microsoft IIS** (Windows-based), **Node.js with Express.js** (JavaScript backend), and **LiteSpeed** (fast and secure commercial server). For example, when visiting a site like YouTube, the web server receives your request,

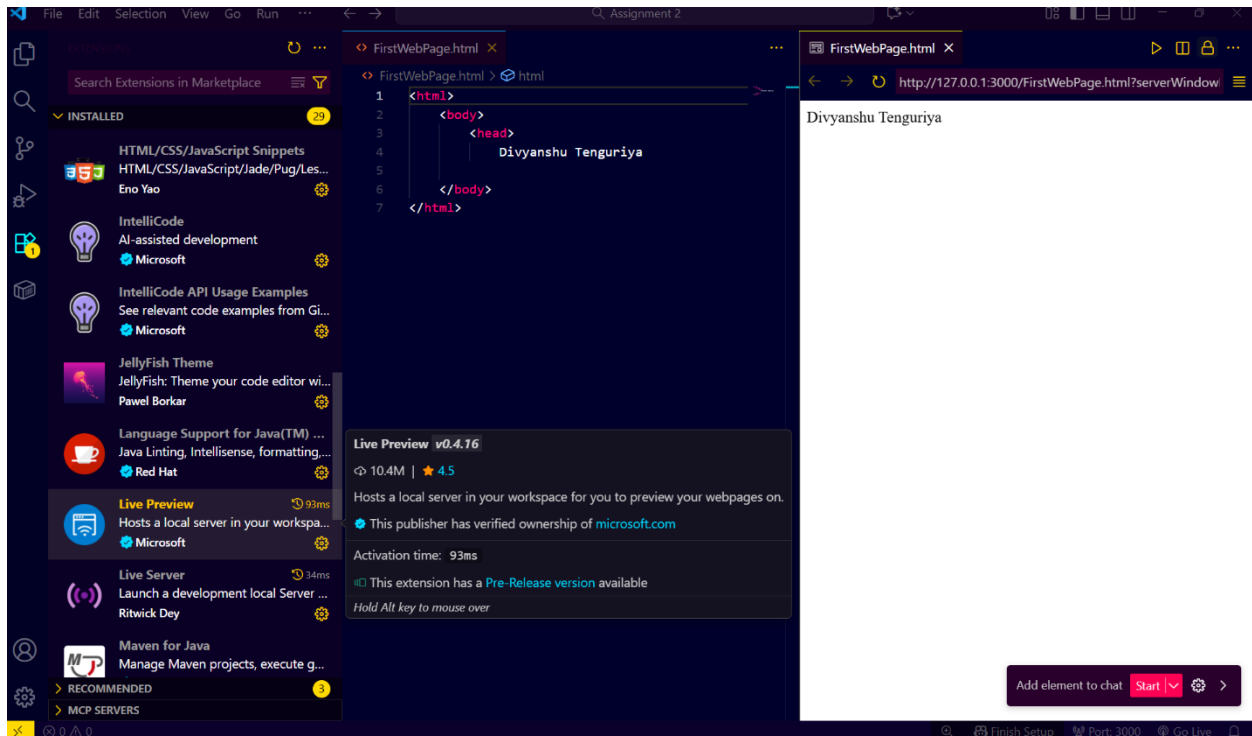
fetches data from the database if needed, and sends back the web page that your browser render.

Q6. Define the roles of a frontend developer, backend developer, and database administrator in a project.

Ans. In a web development project, each role focuses on a different part of the system. A **frontend developer** is responsible for designing and implementing the user interface that users interact with directly. They use technologies like **HTML, CSS, JavaScript**, and frameworks such as **React or Angular** to create layouts, buttons, forms, animations, and ensure a smooth user experience. A **backend developer** works on the server-side logic, handling how data flows between the client and the database. They write code in languages like **Python, Node.js, Java, or PHP**, create APIs, manage server operations, and ensure that requests from the frontend are processed correctly. A **database administrator (DBA)** manages the databases that store the project's data. They design database schemas, ensure data integrity, optimize queries for performance, set up backups, and handle security and access control.

Together, these roles ensure that the web application is **functional, efficient, and user-friendly**: the frontend provides the interface, the backend handles logic and communication, and the database administrator manages the data.

Q 7. Install VS Code and configure it for HTML, CSS, and JavaScript development. Take a screenshot of the setup.



Q 8. Explain the difference between static and dynamic websites. Provide an example of each.

Ans. A **static website** is made up of web pages with **fixed content** that does not change unless a developer manually updates the files. Static sites are built using **HTML, CSS, and sometimes JavaScript** and are simple, fast, and easy to host. An example of a static website is a **personal portfolio website**, where the text, images, and layout remain the same for every visitor. In contrast, a **dynamic website** generates **content on-the-fly** based on user interactions, database queries, or backend logic. Dynamic websites use **server-side languages** like PHP, Node.js, or Python along with databases such as MySQL or MongoDB to display customized content. For example, **YouTube** is a dynamic website because it shows different videos, recommendations, and user data depending on who is logged in and what actions they perform.

Q 9. Research and list five web browsers. Explain how rendering engines differ between them.

Ans. **Five Popular Web Browsers**

- 1. **Google Chrome**
- 2. **Mozilla Firefox**
- 3. **Microsoft Edge**
- 4. **Safari**
- 5. **Opera**

Rendering Engines and Differences

A **rendering engine** is the part of a browser responsible for interpreting **HTML, CSS, and JavaScript** to display web pages correctly. Different browsers use different engines, which can affect how web pages are displayed, their performance, and support for web standards.

Browser	Rendering Engine	Key Differences
Google Chrome	Blink	Fast performance, widely used, excellent support for modern web standards.

Mozilla Firefox	Gecko	Strong standards compliance, open-source, excellent privacy features.
Microsoft Edge	Blink (new version)	Previously used EdgeHTML; now uses Blink like Chrome for better compatibility.
Safari	WebKit	Optimized for Apple devices; energy-efficient and supports Apple-specific web technologies.
Opera	Blink	Uses same engine as Chrome, supports built-in VPN and ad-block features.

Summary

- Browsers interpret web content using **rendering engines**, which convert code into visual web pages.
- Different engines (Blink, Gecko, WebKit, EdgeHTML) may render the same website slightly differently, affecting layout, animations, and performance.
- Choosing a widely supported engine like **Blink or Gecko** ensures better cross-browser compatibility.

Q 10. Draw a labeled diagram showing the basic web architecture flow — client, server database, and APIs.

Ans.

